

Digger's blog

Digger's blog - Author not stated, 4diggers.blogspot.com.

- Published: date not stated
- Original: <<https://4diggers.blogspot.com/>>
- Preserved from: <https://4diggers.blogspot.com/> (stored) on 2026-08-09
- Capture timestamp: 20060615101705
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Digger's blog

Digger's blog

Tuesday, June 06, 2006

How to defeat digg.com

... an introduction to session riding

Are you logged in on digg and not using Opera? Well if you are, you will digg this story either if you like it or not. Anyway, read on and maybe you'll find out some interesting things about session riding.

Session riding is a method for tricking users of websites using cookies for authentication. You probably know that when you login to a website using form authentication, the server will set one or more internal state flags that will tell the server side scripts that you are an authenticated user and let you do things that without authentication you cannot do. For example for PHP those internal state variables are called session variables.

The server will provide you with a session token (usually a big random number encoded in a way) sent to you by a cookie. By that cookie the server can identify you when you are navigating it's pages. It is the only way HTTP protocol can be made stateful. You have this token and by having it you have right to access restricted areas of the website. If a hacker have access to your token then he can do anything he wants in your name (even digg stories for you :)).

The token can be acquired by a hacker in many ways:

- by exploiting XSS flaw on the website
- by sniffing the wire
- by session fixation
- **by session riding**

There are probably other methods which are known but it is not the point of this story.

The problem that is present on digg and **many** (and this is a massive many) sites is that they don't protect their selves against session riding.

**So, what exactly is session riding? **

Imagine the following scenario: you are on your bank's website. You have **logged in** and you are checking your account balance or something. Some "friend" of your's is sending you an email with a link "hey, check out the picture of my new nice and shinny little something". You say "wow, I have to see that", and clicking on the innocent looking link. On the browser page indeed a nice picture appears but what you don't see is that a little hidden iframe is submitting a form to you bank's website (where you have been logged in) and telling to your online banking service that you want to transfer your money to that "friend" of your's who has the new nice and shinny little something. The page is doing that with your session token which has been authenticated. Now your friend have your money too to buy other nice and shinny little somethings.

Now you can say that, "ok, but the server can verify the referer header". You are right but there is a small problem, many users disable referer headers on their browser, saying that "I don't want to be watched by big brother". So a website will choose to verify if the header is correct (coming from the same website, domain, ...) or if it is inexistent. Browsers normally are sending the referer header with each request unless you explicitly disable it.

"Hey, but I didn't disabled referer headers in my browser!" Hm, this is indeed a problem ... well at least if you are using Opera, because any other browser will not send referrer from a frame constructed by JavaScript code.

For example let's take this HTML iframe:

```
<iframe name="myframe" style="width:0px;height:0px;border:0px"></iframe>
```

It is not visible on a page.

Take this javascript function also which will be executed by when a page loads:

```
<script type="text/javascript">
function fillframe() {
mf = window.frames["myframe"];
html = '<form name="getrichform" action="https://secure.mybank.com/transfermoney.php"
method="post">';
html = html + ' <input type="hidden" name="ammount" value="all"/>';
html = html + ' <input type="hidden" name="to" value="my friend bob"/>';
html = html + ' <input type="hidden" name="when" value="right now"/>';
```

```
html = html + '</form>';  
mf.document.body.innerHTML = html;  
mf.document.getrichform.submit();  
}  
</script>
```

This script will fill the iframe with content. For some reason IE and FireFox will not send referer header with any of the links, images, forms, ... etc. included on this frame by the script. On opera it's not working.

Intentionally the attack was demonstrated on a POST request because some people thinks that accepting only POST data will save them from this exploit. Of course with GET it becomes even easier because you can just put an image with the SRC pointing to the page you want to invoke.

You can protect yourself from these kind of attacks in a very easy way. One solution is to use temporary tokens for your links and forms beside the parameters you expect. Those tokens can be included into hidden fields in case of a form or on a parameter value in case of a link. Those tokens should be only valid for one single request. With each response you generate a new one (it can be a single one for all the links, forms or whatever on a page). When a user submits something, you first verify the temporary token and execute actions only if the token matches the value saved on server side. With this method you can be sure that the requesting page is sent by you. Basically you code your own referer header.

Also there are proposals for client side protections but in my oppinion this should be solved by the webserver/webapps and the browsers.

There are some references:

- <http://www.tux.org/~peterw/csrf.txt>
- http://www.isecpartners.com/documents/XSRF_Paper.pdf
- http://www.securenet.de/papers/Session_Riding.pdf
- http://en.wikipedia.org/wiki/Cross-site_request_forgery

posted by [Digger](#) # 12:36 AM 2 comments

Archives

[June 2006](#)

[[image: This page is powered by Blogger. Isn't yours?]](<http://www.blogger.com>)

Archived from <https://4diggers.blogspot.com/>. Rendered offline from the local Markdown copy.